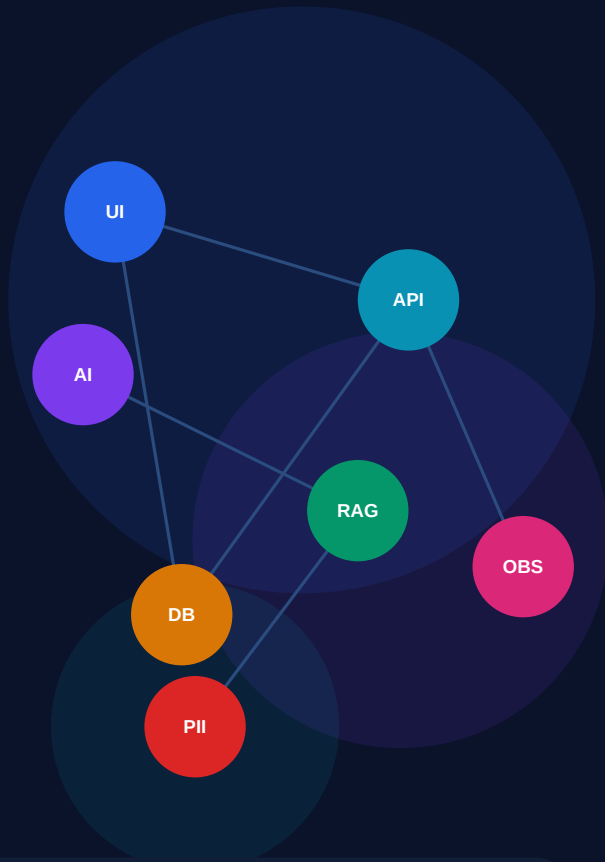
CODE-DERIVED ARCHITECTURE

LawShield

Architecture Atlas

A diagram-first guide to the multi-agent legal AI platform: application flow, LangGraph node topology, Self-RAG control loop, retrieval, privacy, memory, storage, workflows, and operations.



IN THIS ATLAS

01 **Platform map** — Services and ownership

02 **Conversation** — HTTP + SSE lifecycle

03 **LangGraph** — 13 nodes and shared state

04 **RAG** — Ingestion, retrieval, Self-RAG

05 **Trust** — PII, guardrails, citations

06 **Data** — Memory and SQL schema

07 **Workflows** — Documents and templates

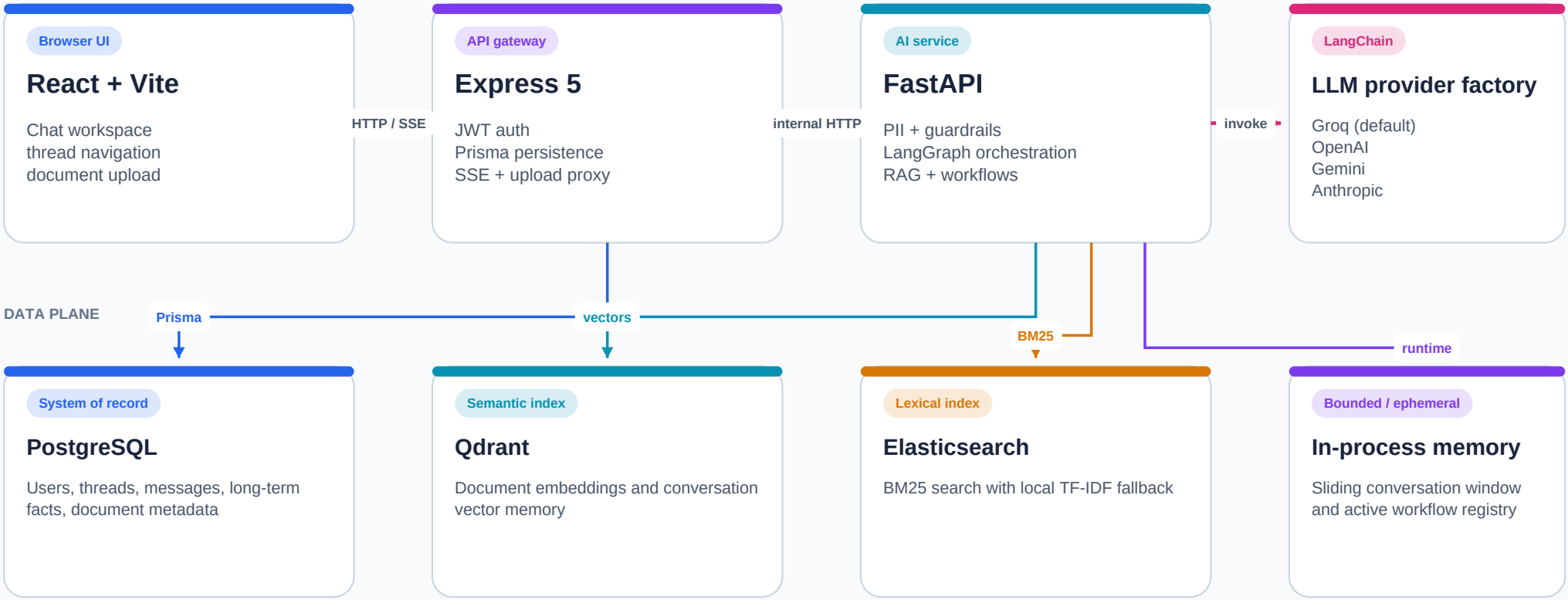
08 **Operations** — Deployment and observability

Architecture snapshot from the repository - 18 August 2026

14 pages

System landscape and service ownership

Three application tiers, three persistence modes, pluggable model providers



Responsibility boundaries

Frontend

- Renders state
- Parses SSE events
- Never calls model vendors

Backend

- Authenticates user
- Checks resource ownership
- Persists durable chat state

AI service

- Masks PII
- Routes and evaluates
- Writes semantic memory

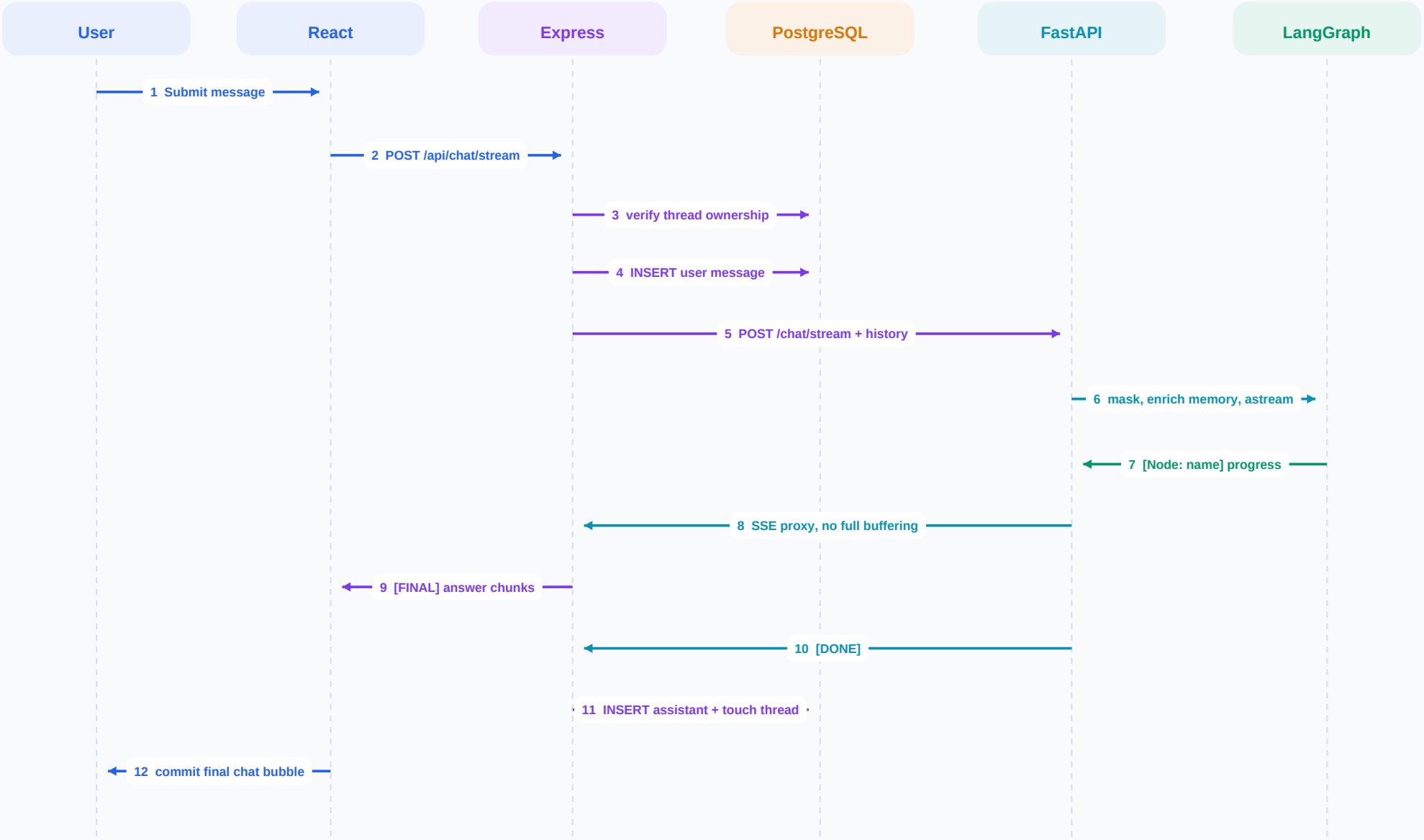
Search stores

- Qdrant: semantic
- ES: keyword precision
- Postgres: relational truth

Sources: README.md; backend/server.js; ai_service/main.py; config/settings.py

Chat request and SSE response lifecycle

Durable message history around a streamed multi-agent execution



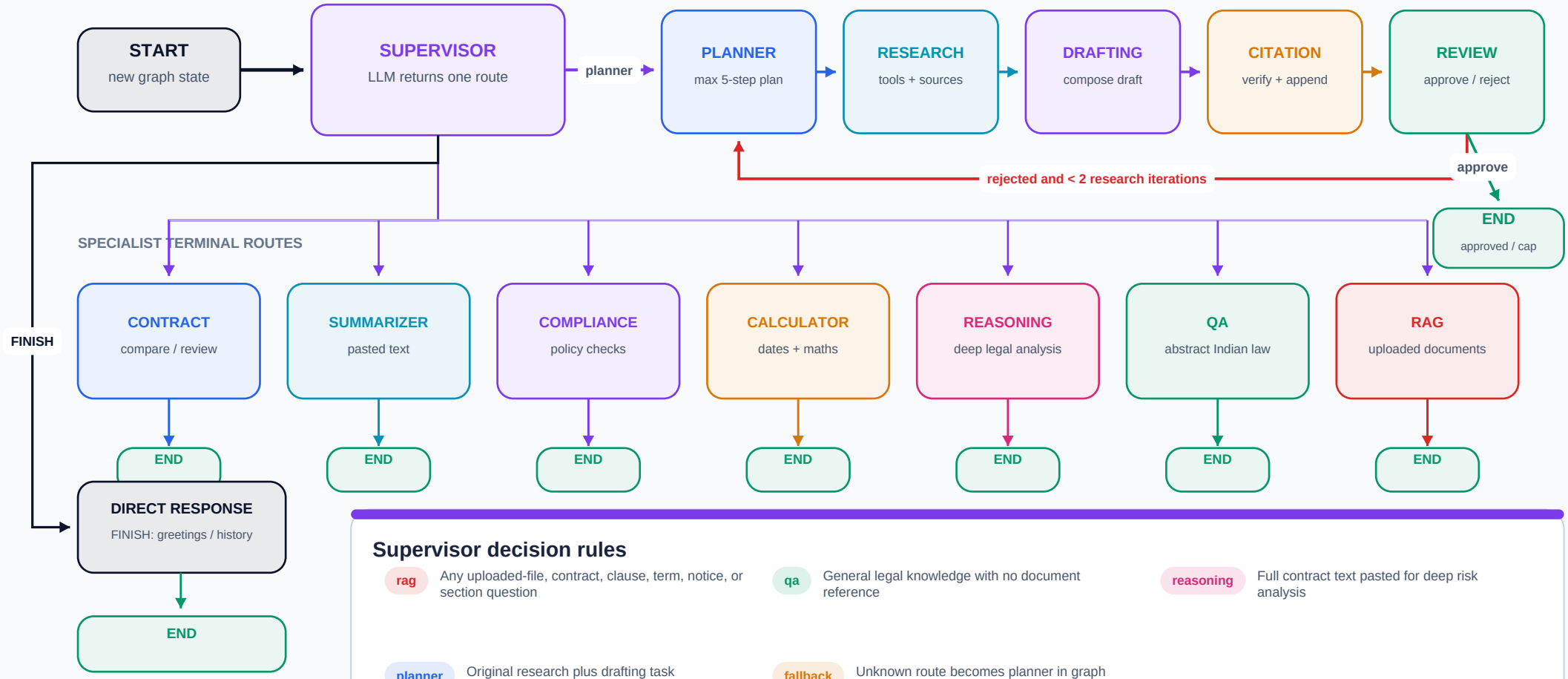
Failure behavior: the user message is persisted before the AI call, so it remains visible if generation fails. The non-streaming endpoint is the frontend fallback.

Trust boundary: ChatService rebuilds masked history, loads long-term and vector memory, validates output, restores PII, then schedules memory writes in background threads.

Sources: frontend/WorkspacePage.jsx; backend/routes/chat.js; ai_service/chat/router.py; chat/chat_service.py

Multi-agent node topology and routing

One supervisor, one iterative research branch, seven terminal specialists



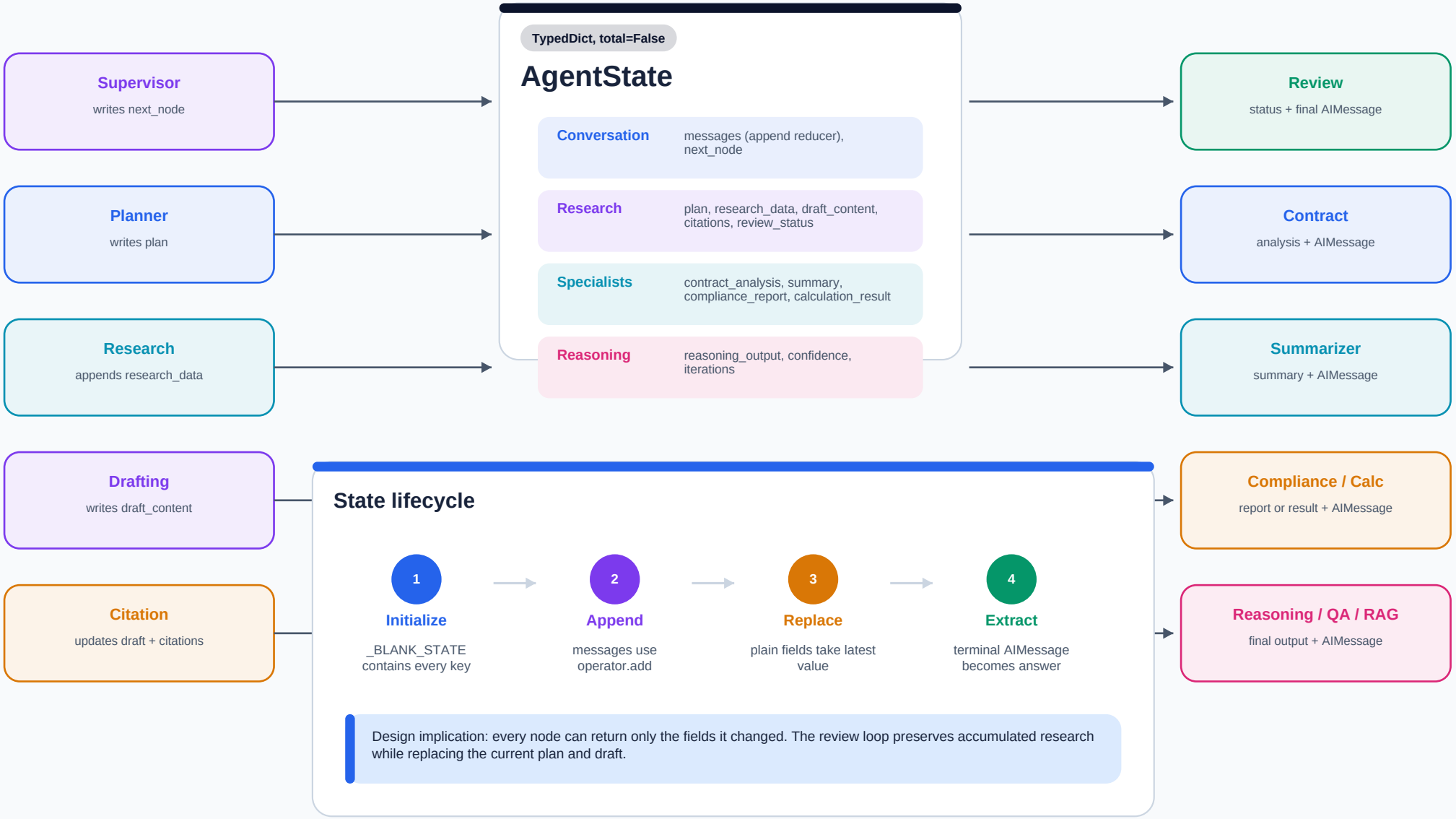
Supervisor decision rules

- **rag** — Any uploaded-file, contract, clause, term, notice, or section question
- **qa** — General legal knowledge with no document reference
- **reasoning** — Full contract text pasted for deep risk analysis
- **planner** — Original research plus drafting task
- **fallback** — Unknown route becomes planner in graph routing; parse failure becomes rag

Sources: ai_service/graph/graph.py; graph/conditions/routing.py; agents/supervisor/agent.py

Shared AgentState and node contracts

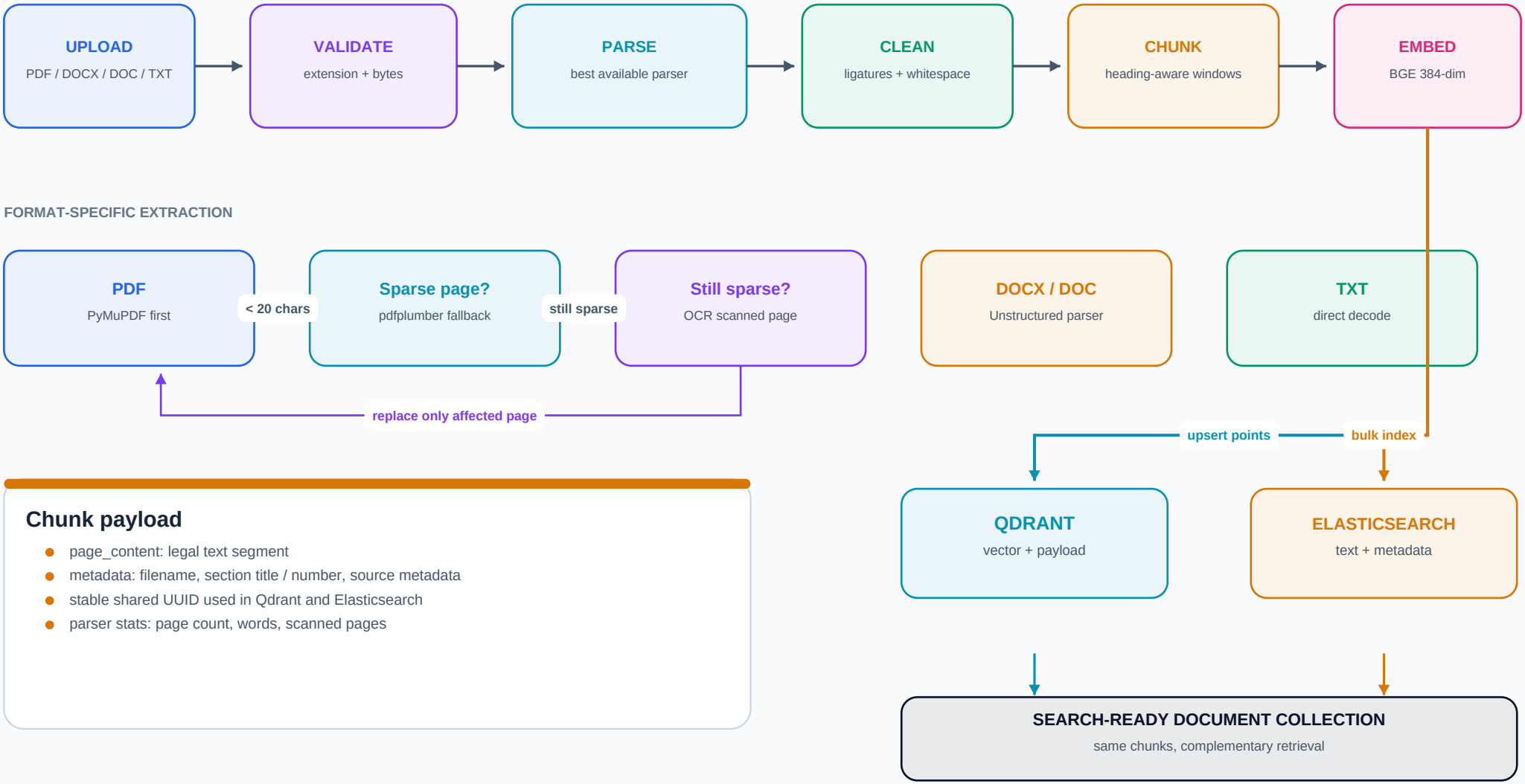
Partial dictionaries merge through a typed state bus; messages append, other fields replace



Sources: ai_service/graph/state.py; agents/*/agent.py; chat/chat_service.py

Document ingestion graph

Parse once, preserve legal structure, index the same chunk IDs for semantic and lexical search



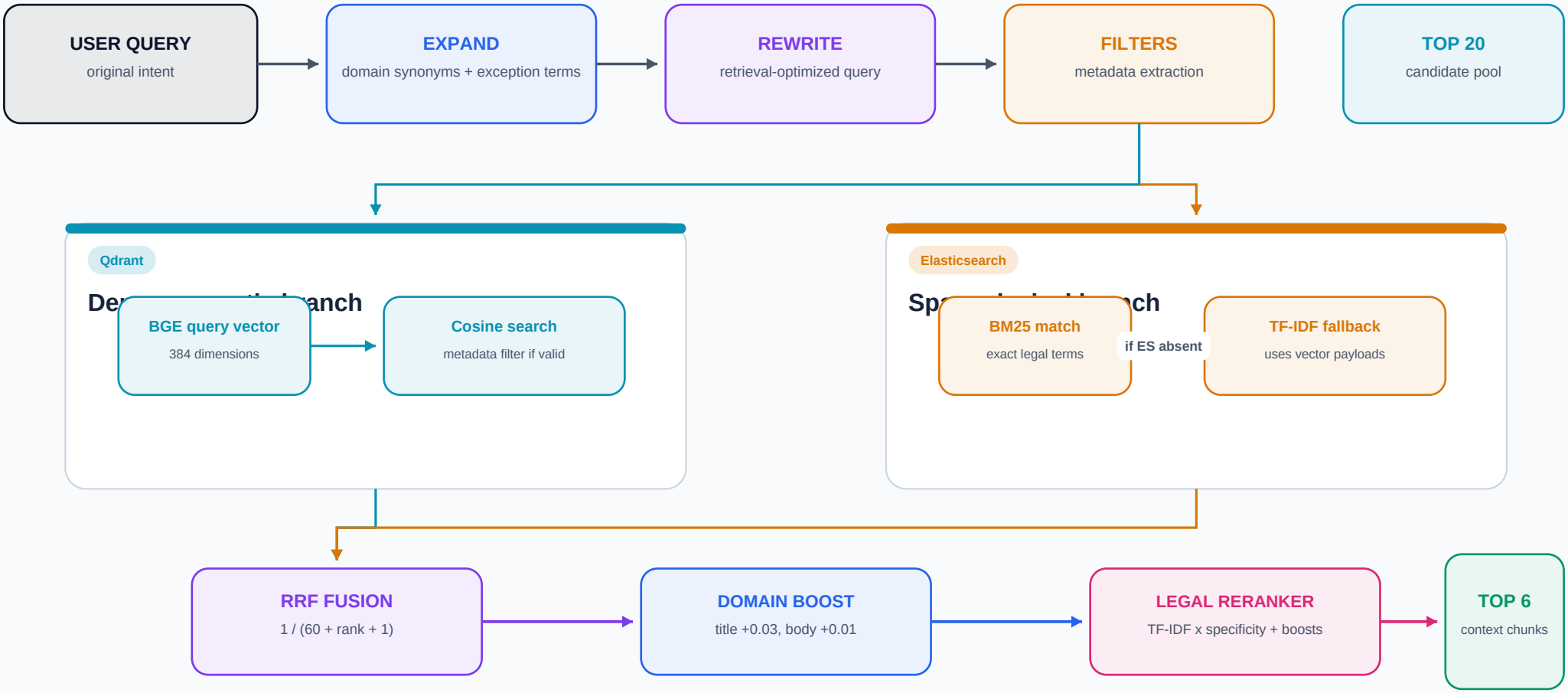
Chunk payload

- page_content: legal text segment
- metadata: filename, section title / number, source metadata
- stable shared UUID used in Qdrant and Elasticsearch
- parser stats: page count, words, scanned pages

Sources: document_processing/loaders.py; document_processing/router.py; rag/chunking.py; rag/pipelines.py

Hybrid retrieval and legal reranking

Semantic recall + keyword precision + exception-aware legal specificity



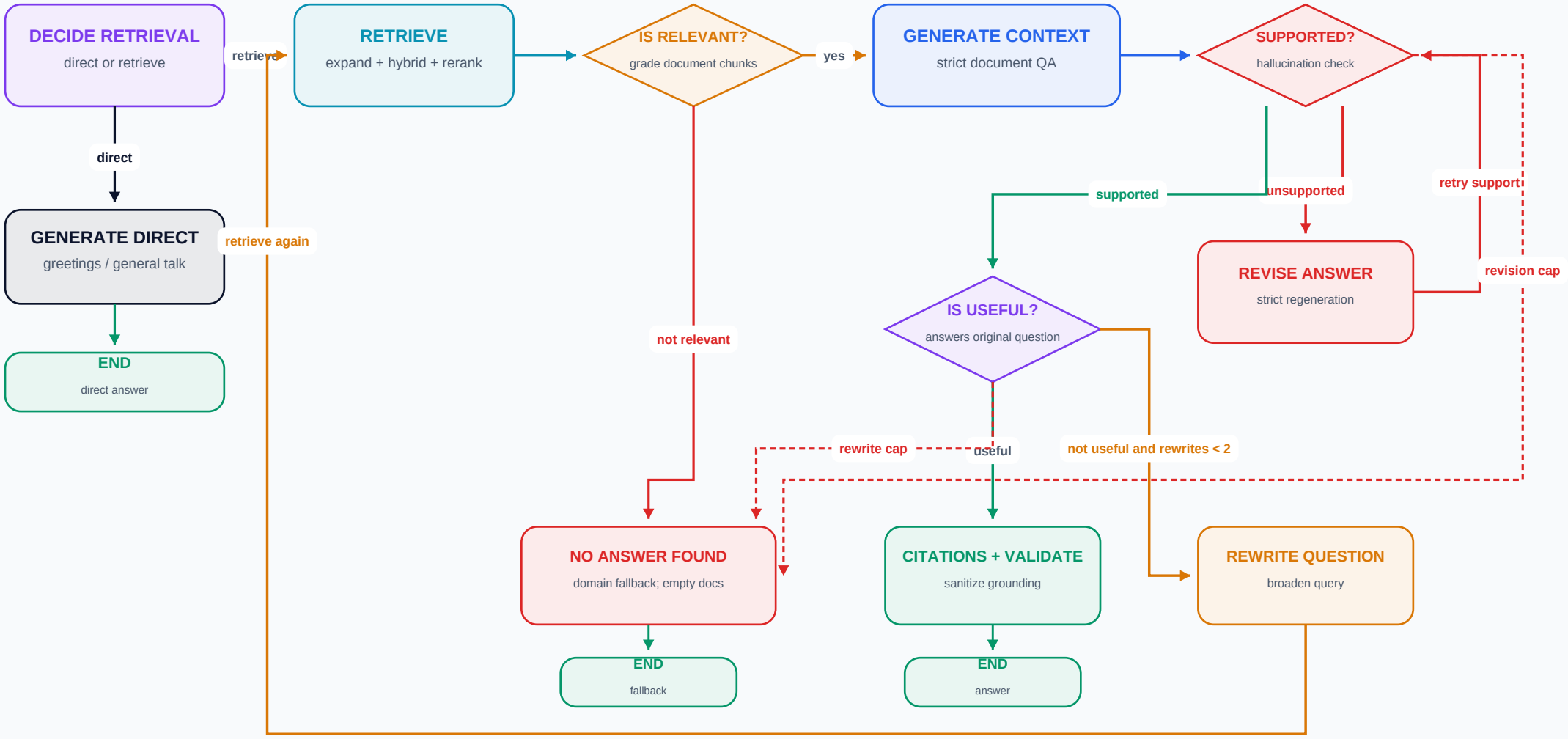
Rerank score favors specific section-title matches and, when the question asks about exceptions, chunks containing terms such as 'unless', 'provided that', and 'subject to'.

Resilience path: filtered Qdrant search retries unfiltered; empty expanded results retry the original question; missing Elasticsearch falls back to local TF-IDF over vector candidates.

Sources: rag/expansion.py; rag/adaptive/query_rewrite.py; rag/hybrid/search.py; rag/reranker.py

Self-RAG control graph

Retrieval decision, relevance grading, grounding repair, usefulness retry, bounded fallback

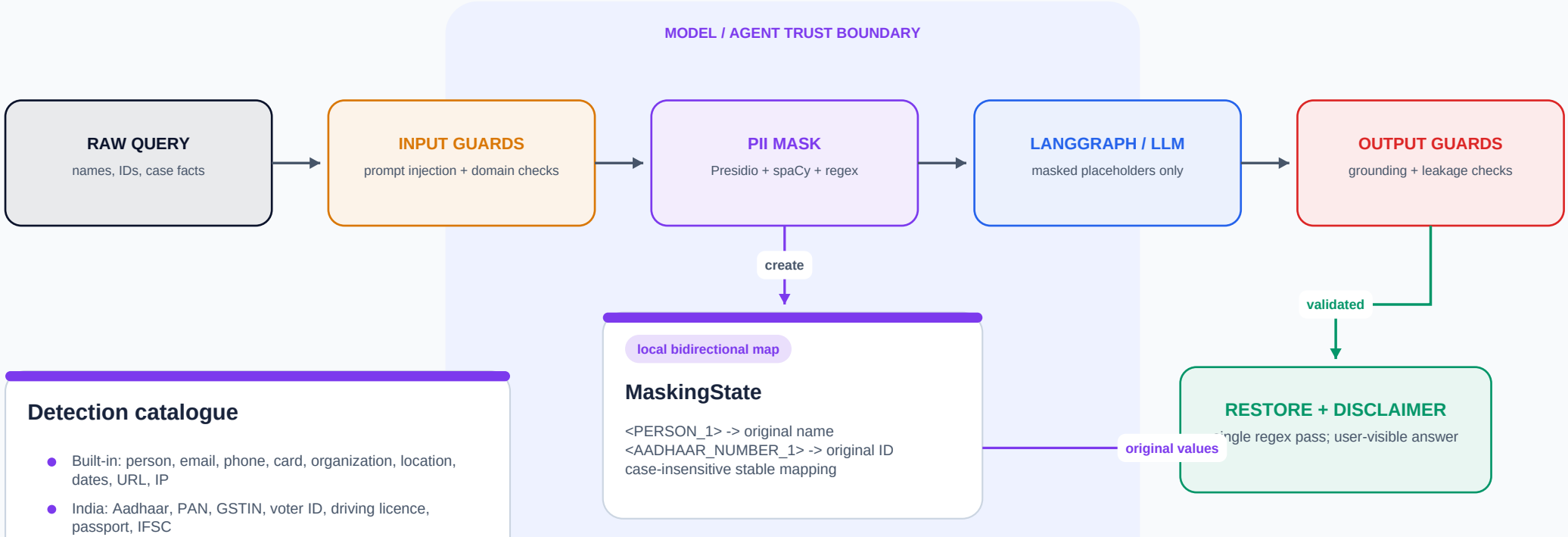


Bounded control state

question | original_question | documents | generation | relevance | support | usefulness | revision_count <= 2 | rewrite_count <= 2 | answer | debug_session

Original identifiers stay behind a local placeholder map while masked text enters model workflows

PII masking, guardrails, and output restoration



Detection catalogue

- Built-in: person, email, phone, card, organization, location, dates, URL, IP
- India: Aadhaar, PAN, GSTIN, voter ID, driving licence, passport, IFSC
- Financial: bank account and IBAN
- Cross-border: US SSN and passport patterns
- Threshold 0.35; overlaps resolved by score and span length

Privacy property: the placeholder map is required to reverse masking, so it remains local to the request/session flow and is not part of the LLM prompt.

Output enforcement

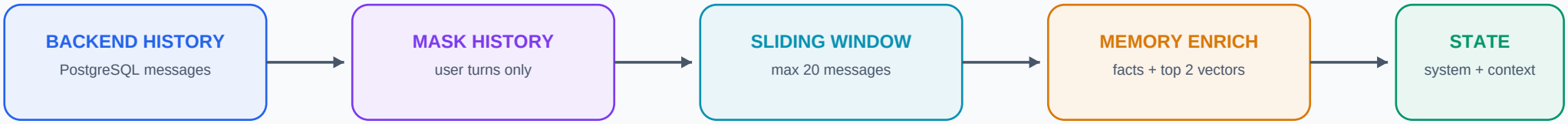
- Verify [Doc N] references against retrieved chunks
- Remove unsupported external statute claims
- Prevent unresolved placeholder leakage
- Restore originals only after validation

Sources: masking/*; legal_guardrails/input_guards.py; output_guards.py; validators.py

Conversation and long-term memory architecture

Durable backend history, bounded prompt context, relational facts, semantic recall

READ PATH - BEFORE GRAPH EXECUTION



durable truth

PostgreSQL

- Full thread and message history
- LongTermFact rows linked to thread
- Fact cap: newest 25 per session

prompt budget

In-process window

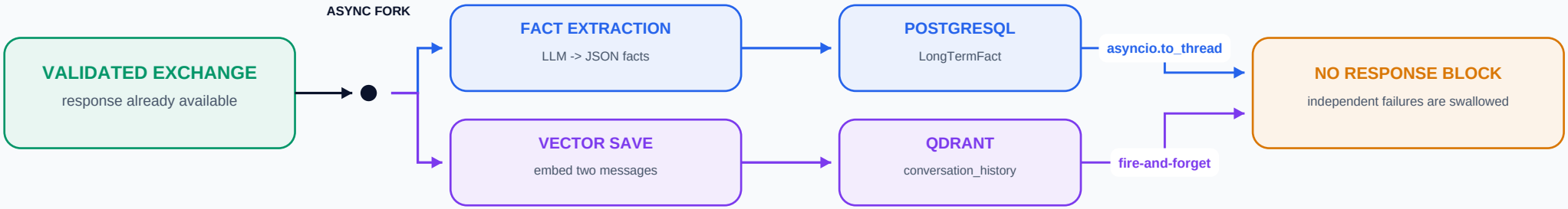
- Rebuilt from backend on every call
- Preserves leading SystemMessage
- Drops oldest non-system turns

semantic recall

Qdrant conversation_history

- 384-dim exchange embeddings
- Payload: session_id + text
- Filtered top-k semantic query

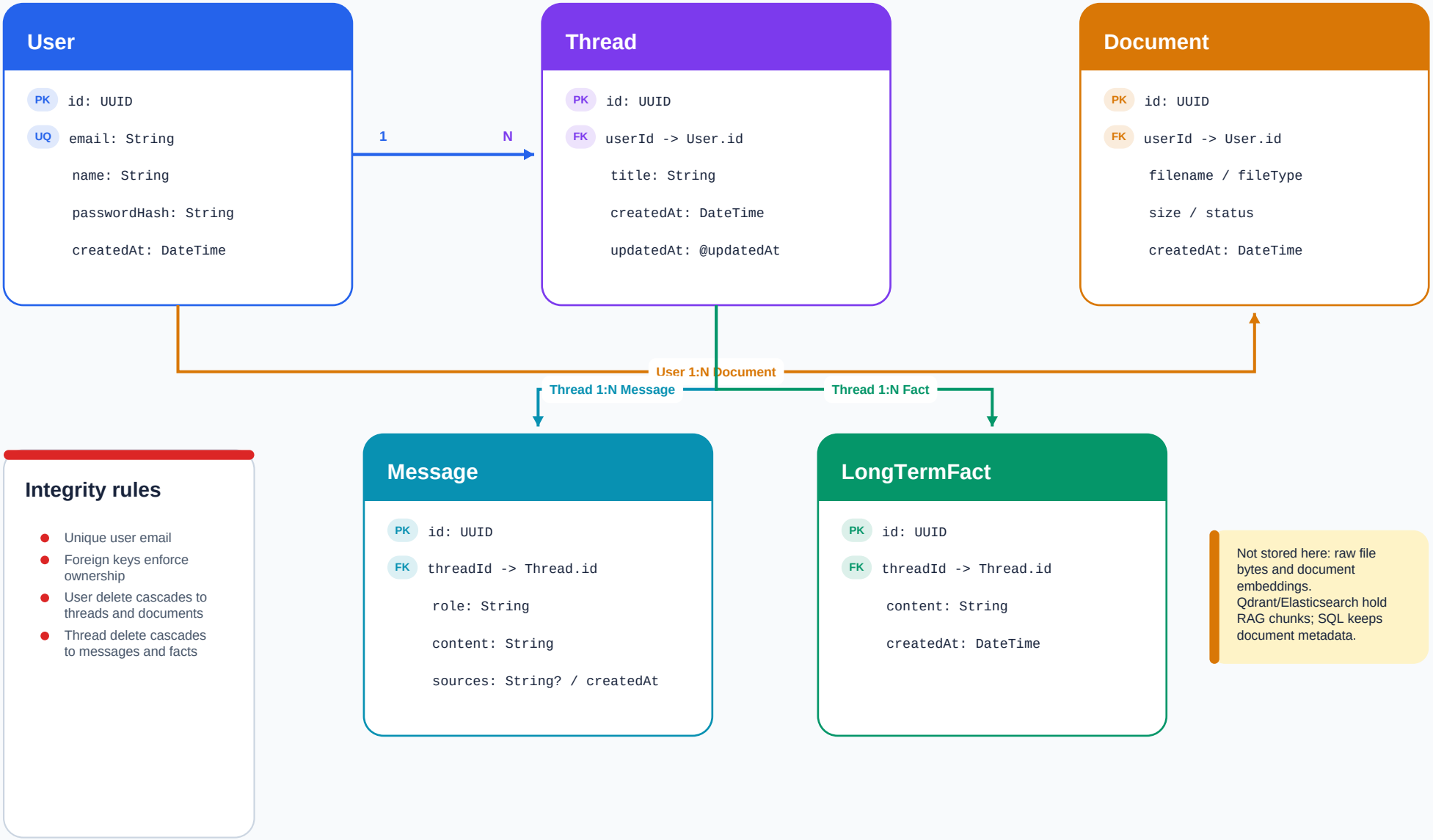
WRITE PATH - AFTER VALIDATED ANSWER



Sources: chat/chat_service.py; memory/conversation.py; memory/long_term.py; memory/vector.py

PostgreSQL relational schema

Identity and ownership constraints for users, matters, messages, facts, and document metadata



Source: backend/prisma/schema.prisma

Upload, processing, indexing, and status flow

A proxy upload path coordinates durable metadata with best-effort AI ingestion

REACT

- multipart upload
- authenticated request

POST

EXPRESS

- Multer memory storage
- 50 MB limit
- MIME allowlist

proxy

FASTAPI

- extension validation
- threaded processing
- partial-success response

index

RAG STORES

- Qdrant vectors
- Elasticsearch text
- shared chunk IDs

PROCESSOR DETAIL

Temp file
guaranteed cleanup

→

Parse
PDF / DOCX / TXT

→

OCR fallback
text-sparse PDF pages

→

Clean + stats
pages, words, metadata

→

Chunk + index
dual RAG write

Success path

1. **AI returns extracted text and ingest stats**
2. **Backend creates Document(status='Ready')**
3. **Frontend refreshes user document list**

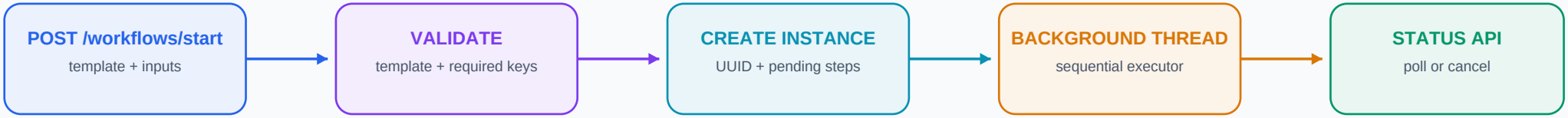
Failure and partial-success paths

- Unsupported type or Multer error -> HTTP 400 / 415
- No extracted text -> FastAPI 422
- Some pages fail -> response keeps text plus error field
- AI ingestion error -> metadata still stored as 'Processing Failed'

Sources: backend/routes/documents.js; document_processing/router.py; loaders.py; rag/pipelines.py

Template workflow engine

A separate sequential background scheduler for repeatable legal deliverables



Execution semantics

Context bus

Step outputs become global_context inputs

Fail fast

Missing inputs or outputs abort the run

Lazy model

LLM initializes on first executed step

Ephemeral registry

Completed/failed runs evict after 1 hour

required: contract_text

Contract review

1. Extract clauses
2. Review clauses
3. Find risks
4. Draft suggestions
5. Generate report

required: research_topic

Legal research

1. Define questions
2. Gather sources
3. Synthesize findings
4. Generate research memo

required: policy_text

Compliance check

1. Identify applicable laws
2. Check compliance
3. Generate gap report

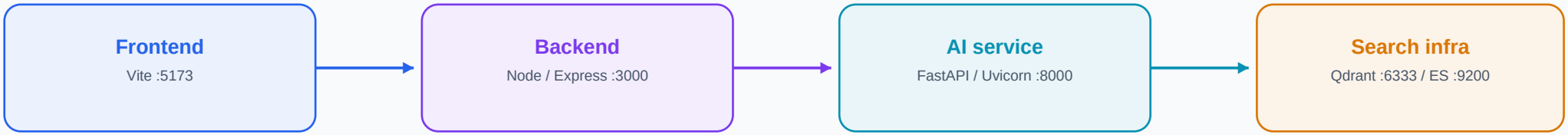
This workflow engine is distinct from the chat LangGraph: it is template-driven, sequential, stored in an in-process registry, and queried through workflow status endpoints.

Sources: workflow/router.py; workflow/templates.py; workflow/scheduler.py; workflow/executor.py

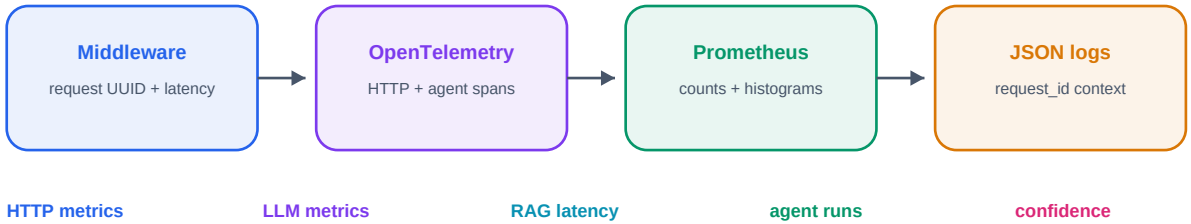
Runtime topology and observability

Local service ports, external dependencies, request tracing, metrics, and structured logs

LOCAL RUNTIME



Request telemetry



HTTP metrics · LLM metrics · RAG latency · agent runs · confidence

Pluggable model layer

Groq	`llama-3.1-8b-instant`	default
OpenAI	`gpt-4o-mini`	optional
Gemini	`gemini-2.5-flash`	optional
Anthropic	`claude-3-haiku`	optional

Architecture in one sentence

LawShield keeps identity and conversation truth in PostgreSQL, semantic evidence in Qdrant, keyword evidence in Elasticsearch, and decision logic in bounded LangGraph workflows protected by masking, grounding checks, and observable request boundaries.

RELATIONAL TRUTH · HYBRID EVIDENCE · BOUNDED LOOPS · PII BEFORE MODEL · STREAMED UX · TRACEABLE OPERATIONS

Sources: start.sh; ai_service/docker-compose.yml; main.py; observability/*; llm/providers/factory.py